

Optimisation des processus de production de séries CVS-CJO

Avec JDemetra+ et R

Anna Smyk and Tanguy Barthélémy (Insee)

Résumé

La désaisonnalisation est un processus clé dans les organismes statistiques publics car elle est nécessaire pour la diffusion et l'interprétation d'un grand nombre d'indicateurs économiques. Dans un souhait d'harmonisation des pratiques, Eurostat recommande depuis 2015 l'utilisation du logiciel *open source* JDemetra+. Cet outil donne notamment accès aux deux algorithmes de désaisonnalisation les plus populaires dans la statistique publique européenne : X-13 Arima et Tramo-Seats, via une interface graphique et une famille de packages R. Cet article vise à décrire et à illustrer comment tirer profit de la version 3.x de JDemetra+ pour construire des processus de production efficaces permettant de corriger plusieurs milliers de séries avec des paramétrages individualisés, tout en contrôlant la qualité statistique et en minimisant l'expertise manuelle.

Mots clés : Désaisonnalisation, statistique publique, processus de production, X13-Arima, Tramo-Seats

JEL Classification: C01, C02, C14, C22, C82, C87

Abstract

Seasonal adjustment is a key process in official statistics because it is necessary for disseminating and interpreting a large number of economic indicators. To harmonize practices, since 2015, Eurostat has recommended the use of JDemetra+. This open-source software provides access, via a graphical user interface and a family of R packages, to the two most popular seasonal adjustment algorithms in European statistical agencies: X-13 ARIMA and TRAMO-SEATS. This article describes and illustrates how to use JDemetra+ version 3 to create efficient production processes that can adjust several thousand series with customized settings while maintaining statistical quality and minimizing manual adjustments.

Keywords: Seasonal adjustment, official statistics, production process, X13-Arima, Tramo-Seats

JEL Classification: C01, C02, C14, C22, C82, C87

Introduction

La désaisonnalisation est un processus clé dans les organismes statistiques publics. La diffusion de la plupart des indicateurs économiques requiert de les purger au préalable des mouvements quasi-périodiques, saisonniers et de calendrier, afin de faciliter l'interprétation des évolutions de court terme, le repérage des points de retournement et des tendances moyen et long terme (Bell et Hillmer 1984). Cependant, la mise en place d'un processus de production de séries corrigées des variations saisonnières et des jours ouvrables (CVS-CJO) à la fois standardisé, flexible et respectant les bonnes pratiques du système statistique européen (Eurostat 2024) s'avère coûteux. Au delà de la production de séries désaisonnalisées celui-ci doit également fournir des indicateurs de qualité intégrés (Kirchner et alli 2018) et permettre un ajustement manuel par des experts, souvent soumis à de fortes contraintes de temps. A l'Insee et au sein du Système Statistique Public (SSP) la quasi totalité des processus de désaisonnalisation est repose sur JDemetra+ (Smyk et alli 2026), officiellement recommandé par Eurostat depuis 2015. Ce logiciel, *open source*, pour l'analyse des séries temporelles donne accès aux deux algorithmes de désaisonnalisation les plus populaires dans la statistique publique européenne : X-13 Arima (US-Census-Bureau 2015) et Tramo-Seats (Gomez et Maravall 1996). Les algorithmes sont écrits en Java et accessibles via une [interface graphique](#) et une famille de packages R, le [rjdverse](#).

La version 3 de de JDemetra+, disponible depuis 2023, offre de nouveaux outils R pour optimiser les processus de production en automatisant un plus grand nombre de tâches. De plus, la crise sanitaire de la Covid-19 a mis en évidence la nécessité de paramétrages fins à très grande échelle, comme l'attribution d'un profil d'outliers spécifiques à chaque série pour plusieurs centaines de séries. Enfin, l'accent mis ces dernières années sur le passage de SAS à R à l'Insee favorise l'adoption de ces nouveaux outils dans un cadre de général de modernisation. Le contexte actuel est donc très favorable pour revoir et optimiser les chaînes de production de séries CVS-CJO existantes.

Le Département des Méthodes Statistiques de l'Insee est responsable chaque année de plusieurs refontes de processus, diffuse les bonnes pratiques et forme les producteurs au logiciel JDemetra+. Toutefois, pour clarifier la construction et l'optimisation des processus de désaisonnalisation, un exemple de méthode et une présentation des outils les plus récents s'avère nécessaire.

En effet, actuellement, le producteur de séries CVS-CJO dispose éléments théoriques sur les algorithmes d'ajustement saisonnier, comme le [Handbook on Seasonal Adjustment](#), d'un guide de [bonnes pratiques](#), de la [documentation du logiciel JDemetra+](#) et éventuellement des notes reflétant la sédimentation historique des pratiques dans son institution. Une pièce nous semble manquer dans ce puzzle : un exemple précis d'implémentation avec les outils les plus à jour. C'est l'objet de la suite de cet article qui vise à décrire et à illustrer comment tirer parti de JDemetra+ version 3 pour pour corriger jusqu'à plusieurs milliers de séries avec des paramétrages individualisés tout en contrôlant la qualité statistique et ciblant l'expertise manuelle.

La production de séries désaisonnalisées comporte généralement trois temps : la mise en place du processus, phase de fortes évolutions méthodologiques, ainsi que les campagnes de révision annuelles et infra-annuelles. Les processus de production peuvent être basés sur une structure de données spécifique à JDemetra+ (*workspaces*), permettant d'utiliser l'interface graphique, ou être construits entièrement en R.

Les campagnes annuelles nécessitent des outils pour la comparaison de modèles et la priorisation des séries nécessitant une expertise manuelle, ce qui peut être fait avec le package R JDCruncher. La production infra-annuelle requiert des politiques de révision des données et des outils d'analyse des révisions pour valider les chiffres avant la publication. Nous détaillons ces trois phases en montrant les avantages et inconvénients de différentes configurations, en présentant les outils disponibles et en comparant les temps d'exécution. Nous avons joint des extraits de code R dans l'article et un exemple complet est disponible dans la [vignette du package {rjd3production}](#) .

Après une brève présentation des opérations statistiques d'un processus de désaisonnalisation et du logiciel de JDemetra+, nous détaillons dans une deuxième partie la construction d'une chaîne de production. La troisième partie est consacrée aux campagnes annuelles et infra-annuelles de révisions des paramètres d'estimation. En annexes, nous fournissons une présentation plus détaillée des outils utilisés.

Partie 1 : Désaisonnalisation et production

Le processus statistique

L'ajustement saisonnier permet de purger les séries temporelles des mouvements périodiques qui masquent les variations conjoncturelles et les tendances de long terme. Pour cette raison, les indicateurs économiques sont pour la plupart publiés corrigés des variations saisonnières (CVS) et des effets de calendrier (CJO). Désaisonnaliser nécessite d'estimer une composante saisonnière notée S capturant l'ensemble des mouvements infra-annuels périodiques afin de l'enlever de la série brute $Y_{\text{CVS}} = Y - S$. Pour ce faire, on décompose la série brute en trois composantes inobservables : saisonnalité, tendance et irrégulier ($Y = S + T + I$), où la tendance T représente les mouvements de long terme et I le bruit engendré par des événements exceptionnels (grèves, conditions climatiques, chocs de court terme...) et l'ensemble des erreurs de mesure. Les algorithmes les plus utilisés actuellement au sein des instituts nationaux de statistique européens et des banques centrales sont Tramo-Seats (Gomez et Maravall 1996) et X13-Arima (US-Census-Bureau 2015). Ce dernier étant quasi exclusivement utilisé au sein du service statistique public français. Ces méthodes procèdent en deux étapes : la série brute n'est pas directement décomposée mais linéarisée au préalable, en enlevant (définitivement) les effets de calendrier et (temporairement) les points atypiques en utilisant une modélisation Reg-Arima.

Objectifs

L'objectif principal est de produire des séries CVS-CJO, notamment en vue d'une publication. Pour s'assurer de la qualité de la désaisonnalisation un ensemble plus large d'indicateurs est cependant nécessaire. Ainsi, le processus doit permettre de visualiser les diagnostics (tests de saisonnalité, tests de saisonnalité résiduelle et d'effets de jours ouvrables résiduels pour la série désaisonnalisée, etc.) et d'agir sur les paramètres au vu des problèmes pointés par ceux-ci. Les séries intermédiaires, en plus de la série CVS-CJO finale, comme les composantes inobservables, la série linéarisée et les facteurs saisonniers, y compris prévus, s'avèrent souvent utiles.

Etapes

Un processus de désaisonnalisation commence par un test de saisonnalité pour s'assurer que cette opération est bien nécessaire sur la série considérée, et si ce n'est pas le cas la série brute sera aussi diffusée comme CVS-CJO. Dans la pratique on réalise souvent ce test en fin de processus, sur la série linéarisée (dont seront extraits les facteurs S). Cela permet de garder toujours le même jeu de données en entrée, en excluant seulement au stade de la diffusion les séries à ne pas désaisonnaliser.

Une correction des effets de calendrier doit souvent être implémentée ou du moins testée¹. On peut ensuite procéder à la première estimation (pré-ajustement et décomposition), avec dans la plupart des cas une spécification largement automatique et générer un bilan qualité qui permettra d'améliorer la qualité de la désaisonnalisation. On parle ici de bilan qualité et pas simplement de diagnostics, car celui-ci consiste en une mise en forme des diagnostics, et une synthèse en un score qui permet une approche de *selective editing* pour prioriser l'expertise manuelle des séries. Il est en général nécessaire de quantifier les révisions et l'impact numérique du choix des paramètres, qui vont intervenir dans le choix des paramètres finaux. Une fois ces éléments revus une deuxième estimation peut avoir lieu, le processus est éventuellement répété jusqu'à converger vers un jeu final de paramètres.

Table 1: Les étapes

Domaine	Opérations
Correction des effets de calendrier	Création d'un calendrier de référence Génération des jeux régresseurs Sélection du meilleur jeu pour chaque série
Personnalisation des spécifications	Attribution des régresseurs de calendrier Attribution des outliers Autres paramètres
Estimation	Estimation Préliminaire Production output

¹Les guidelines d'Eurostat recommandent de ne corriger les effets de calendrier ayant un sens ex-ante.

Domaine	Opérations
Contrôle Qualité	Production d'un bilan qualité Expertise manuelle Estimation finale Production output final

Le processus de production

Propriétés

La production nécessite par nature d'automatiser un maximum d'opérations (définition des spécifications, réalisation des estimations, production d'un rapport qualité, application des politiques de révision) mais dans aussi, dans le cas de la désaisonnalisation, de visualiser les diagnostics et graphiques dans un environnement permettant un ajustement manuel rapide des paramètres.

Le processus peut être scindé en trois phases distinctes par leur objectifs et le type d'intervention qu'elles requièrent : mise en place d'un processus de production, campagnes annuelles et infra-annuelles.

Mise en place

Il s'agit d'initialiser le processus en faisant des choix de moyen terme qui ne seront *a priori* pas remis en cause pendant une période d'environ cinq ans. Les besoins des utilisateurs sont alors spécifiés : on sélectionne les variables d'intérêt à désaisonnaliser, le niveau de la nomenclature ainsi que l'approche directe (désaisonnalisation à chaque niveau) ou indirecte (agrégation de séries désaisonnalisées), voir Guidelines, chapitre 5.4 pour plus de détails. La pertinence d'une correction des effets de calendrier doit être testée. C'est également le moment de déterminer la longueur de la période d'estimation (*Ibid.*, chapitre 8) et la profondeur des révisions lors d'une publication (*Ibid.*, chapitre 6). Lors de cette initialisation, on choisit également aussi l'algorithme et le logiciel utilisés.

Campagnes annuelles et infra-annuelles de révision

Environ une fois par an (*Ibid.*, chapitre 6), le producteur fait une mise à jour des paramètres statistiques au vu des nouvelles données brutes. Une manière de procéder peut-être la comparaison entre les modèles "en vigueur" et une ré-estimation automatique, comme décrit dans la [partie 3](#) .

Lors de la production [infra-annuelle](#) , on cherche à minimiser les révisions techniques. Il est recommandé de ne revoir qu'un nombre limité de paramètres et même courant d'utiliser les

coefficients saisonniers prévus lors de la dernière campagne annuelle. On se focalise sur les révisions et sur l'apparition ou disparition de points aberrants dans la période récente.

La suite de cet article présentera une procédure ainsi que les outils nécessaires pour implémenter les trois phases décrites ci-dessus. Dans l'objectif d'harmoniser les pratiques et les outils statistiques, JDemetra+ est recommandé par Eurostat au pays membres du système statistique Européen pour l'ajustement saisonnier depuis 2015. Nous supposons donc l'utilisation de ce logiciel dans la suite. La section suivante en présente brièvement les principales caractéristiques relatives à la production de séries CVS-CJO.

JDemetra+ : un outil pour l'ajustement saisonnier

JDemetra+ est une bibliothèque d'algorithmes *open source* pour l'analyse des séries temporelles. Ceux-ci sont accessibles via une interface graphique (GUI²), éventuellement enrichie par des [plug-ins](#) et un écosystème de packages R, le [rjdverse](#). En plus de la désaisonnalisation, ils permettent également de faire du nowcasting, du benchmarking, de la désagrégation temporelle, de l'estimation de tendance et l'analyse des révisions. Le développement et la promotion de JDemetra+ sont co-financés par Eurostat à travers des *grants* successifs et actuellement au sein du projet [COSA](#).

L'ajustement saisonnier est le domaine pour lequel JDemetra+ est le plus connu. Ce logiciel a en effet d'abord été conçu comme une simple interface graphique, s'appuyant sur les implémentations originales des deux algorithmes X12-arma et Tramo-Seats. L'objectif était d'offrir à l'utilisateur une visualisation conviviale des résultats, diagnostics et paramètres, ainsi que d'harmoniser celle-ci pour les deux algorithmes. Le logiciel s'est d'abord appelé [Demetra](#), déesse grecque de l'agriculture et des saisons. Lorsque les programmes en Fortran de l'US Census Bureau et de la Banque d'Espagne ont été ré-écrits en Java³, Demetra est devenu JDemetra+ (Grudkowska et al. 2013).

Algorithmes

Désormais, JDemetra+ propose des algorithmes supplémentaires pour l'ajustement saisonnier, mais seuls les deux historiques sont réellement conçus pour la production massive de données, dans la mesure où ils permettent de stocker les paramètres dans un [workspace](#) et d'utiliser un module d'automatisation le [Cruncher](#). Dans nos exemples nous ferons référence à X13-Arima, mais l'ensemble des recommandations reste valable avec Tramo-Seats.

²Graphical User Interface

³Les algorithmes sous-jacents sont identiques mais les procédures d'estimation pas nécessairement, ce qui peut donner lieu à des résultats numériques légèrement différents.

Table 2: Algorithmes d’ajustement saisonnier

Algorithme	GUI	Packages R	Adaptés à la production
X13-ARIMA	✓	{rjd3x13}	oui
Tramo-Seats	✓	{rjd3tramoseats}	oui
X12+	✓	{rjd3x11plus}	non
STL	✓	{rjd3stl}	non
Basic Structural Models	✓	{rjd3sts}	non

Tous les algorithmes ci-dessus permettent de désaisonnaliser des données mensuelles, trimestrielles mais aussi bimestrielles, quadrimestrielles et bi-annuelles. Ils disposent également d’extensions pour les données haute-fréquence (infra-mensuelles) détaillées dans Smyk et Webel (2024).

Outils

Interface graphique (GUI)

JDemetra+ est disponible sous la forme d’une [interface graphique](#) destinée à faciliter l’expertise manuelle grâce à une présentation organisée des résultats et paramètres. Les manipulations reposent essentiellement sur du clic-bouton, ce qui nécessite des outils auxiliaires d’automatisation. L’utilisation de la GUI, et du *Cruncher*, exige également d’organiser ses données en une structure spécifique : le [workspace](#).

Cruncher

Le *Cruncher* est un module exécutable en ligne de commande qui permet de mettre à jour un *workspace* sans avoir à ouvrir l’interface graphique. Ce module permet de :

- rafraîchir les données (les mettre à jour partir des fichiers de données brutes)
- [rafraîchir les modèles](#) d’ajustement saisonnier selon une politique de rafraîchissement
- produire des outputs (séries, diagnostics et paramètres).

Nou utiliserons par la suite ce module avec deux packages R, {rjwsacruncher} pour les estimations et l’export des résultats et {JDCruncheR} pour la mise en forme des diagnostics. L’avantage du *Cruncher* est aussi sa rapidité, il peut être exécuté sur des centaines /milliers de séries et produire les résultats en un [temps raisonnable](#). La procédure d’installation et les principales fonctions sont décrites dans la [documentation](#).

Packages R

Une famille de packages, le [rjdverse](#)  donne, d'une part, directement accès en R aux algorithmes de JDemetra+ et permet d'autre part d'automatiser des opérations relatives à l'utilisation des workspaces. Le package `{rjd3production}` offre quant à lui des macro-fonctions correspondant à des étapes complètes de la chaîne de production.

Table 3: Packages R utilisés pour la production de séries CVS-CJO

Package	Fonctions utiles pour la production
{rjd3toolkit} 	Génération de régresseurs de calendrier, Personnalisation des spécifications, Tests
{rjd3x13} 	Désaisonnalisation (Pré-ajustement Reg-Arima et décomposition X11)
{rjd3tramoseats} 	Désaisonnalisation (Pré-ajustement TRAMO et décomposition SEATS)
{rjwsacruncher} 	Mise à jour du <i>workspace</i> et export des outputs
{JDCruncherR} 	Créer un bilan qualité pour le <i>workspace</i> à partir des outputs
{rjd3production} 	Wrapper avec macro-fonctions par étape de production
{rjd3workspace} 	Création et modification des workspaces
{rjd3providers} 	Gestion des données en entrée

Les procédures d'installation de ces outils se trouvent dans la [documentation](#) .

Structures de données

Dans les paragraphes suivants, nous décrivons brièvement les structures de données et concepts propres à JDemetra+, ceux-ci seront utiles dans les parties suivantes.

Les séries en entrée

L'interface de JDemetra+ accepte différents types de formats de données que l'on importe à partir des menus de l'onglet [providers](#) .

- Des fichiers CSV (en précisant le séparateur de champ, le format pour la date, le délimiteur des décimales...)
- Des fichiers Excel (uniquement *.xlsx en version 3.x)
- Des fichiers XML
- Des sources de données SDMX
- Des bases de données ODBC ou JDBC

Toutes les séries, que ce soient les données à désaisonnaliser ou les variables auxiliaires sont importées de la même façon. Pour les estimations, les packages R travaillent eux avec des objets de classe TS.

Outputs

JDemetra+ génère différents types d'outputs :

- Des séries (désaisonnalisée, facteurs saisonniers, série linéarisée)
- Des paramètres (informations sur les spécifications)
- Des diagnostics (informations sur la qualité de l'ajustement)

L'interface graphique permet de visualiser ces outputs et de les exporter à l'aide de [menus déroulants](#) .

Le *Cruncher* évite la navigation dans les menus en proposant un export identique de manière automatisée. Les paramètres et les diagnostics sont regroupés en un fichier de données individuelles où l'identifiant est le nom de la série. Le fichier exporté s'appelle *demetra_m.csv*, "m" pour matrice.

Si l'estimation est faite en R, par exemple avec `{rjd3x13}`, une liste de listes regroupe les séries, diagnostics et paramètres.

```
library("rjd3x13")
mod <- x13(my_series)
mod$result$final
```

Spécifications

Une spécification est l'ensemble des paramètres, contraintes propres à X-13 ou Tramo-Seats, permettant de désaisonnaliser une série. Chaque algorithme a un jeu de [spécifications par défaut](#)  que l'utilisateur peut personnaliser. Ainsi, une spécification sera attachée à chaque série, stockée dans le *workspace* ou dans l'objet R.

Plus précisément, la version 3 de JDemetra + permet de manipuler trois versions d'une même spécification de départ: Reference Specification, Estimation Specification et Result Specification. La distinction entre ces trois stades devient utile lorsque l'on applique une politique d'estimation ou de rafraîchissement avec la GUI ou le *Cruncher* lors de campagnes annuelles ou infra-annuelles.

- **Estimation Specification** : Ensemble des contraintes utilisées par l'algorithme pour mener à bien les calculs et obtenir les séries et les indicateurs attendus. Si l'utilisateur n'a modifié aucun paramètre elle sera égale à l'une des spécifications par défaut.

- **Result Specification** : Résultat complet de l'estimation (coefficients du modèle Reg-Arima, longueur et poids des filtres..) permettant de retrouver les séries estimées à partir de la série brute. Tous les paramètres y sont déterminés, il n'y a plus d'identification ou de recherche automatique d'effets. Cette spécification peut-être appliquée à une série (compatible).
- **Reference Specification** : Correspond au départ à la spécification par défaut sélectionnée par l'utilisateur. Lors d'une politique de rafraîchissement elle remplace les paramètres de l'**Estimation Specification**, même si celles-ci ont été spécifiées par l'utilisateur, là où les contraintes sont relâchées. Si l'on veut voir un paramètre de **Estimation Specification** préservé alors que la politique de rafraîchissement est susceptible de le re-identifier ou re-estimer, il doit être dans la **Reference Specification**.

L'affichage et les modifications par l'utilisateur de ces trois types de spécifications ainsi qu'un exemple de cas d'utilisation sont présentés en [annexes](#) .

La définition des spécifications et le comportement des refresh policies est différent de JDemetra+ version 2, nous donnons des détails en [annexes](#) .

Workspaces

Un *workspace* est une structure de données spécifique à JDemetra+, composée d'un ensemble de fichiers XML organisés en dossiers (image et lien vers doc) contenant, entre autres :

- Les variables externes (régresseurs de calendrier...)
- Les calendriers personnalisés du *workspace*
- Les spécifications (modèles d'ajustement saisonniers)
- Des SA-Processing (conteneur de séries et de modèles)

Au premier niveau, un *workspace* est l'association d'un répertoire (comportant des fichiers XML) et d'un fichier XML maître qui définit la structure du dossier et de son contenu, les deux portent le même nom et doivent figurer ensemble dans même répertoire.

Un *workspace* contient des SA-Processing, l'équivalent de sous répertoires propres à JDemetra+, c'est là que sont créés des SA-items, sous structures contenant toutes les informations relatives à une série (données brutes, liens vers celles-ci et les trois types de spécifications vues dans la section précédente. Le *workspace* ne stocke pas de résultats mais l'ensemble des paramètres nécessaires pour les régénérer. Nous complétons cette description en [annexes](#) .

La partie suivante détaille les étapes de la construction d'une chaîne de production en combinant les outils décrits ci-dessus.

Partie 2 : Construction d'une chaîne de production

Deux approches

JDemetra+ permet d'utiliser les algorithmes de désaisonnalisation via des packages R ou l'interface graphique (GUI). L'utilisation de la GUI est généralement motivée par une étape ultérieure d'expertise manuelle, celle-ci sera en effet grandement facilitée par une vue organisée des résultats et des diagnostics ainsi que la possibilité de modifier les spécifications de manière guidée. L'utilisation de la GUI ne contraint toutefois pas l'utilisateur à effectuer toutes les opérations manuellement avec la souris. Deux voies d'automatisation existent : le *Cruncher* permet de lancer les estimations et d'exporter les outputs, alors que les packages {rjd3production} et {rjd3workspace} permettent d'implémenter à grande échelles des opérations telles que la personnalisation des spécifications.

Les producteurs recourant peu ou pas à l'expertise manuelle sont de plus en plus enclins à utiliser directement les fonctions R et ne manipulent alors plus que des objets TS (Time Series) en s'affranchissant de la structure de *workspace*. Ces deux approches ne sont toutefois pas exclusives, il est tout à fait possible de recréer un *workspace* fonctionnel (lisible par la GUI et crunchable) à partir d'objets R.

Dans la suite de cette partie nous nous plaçons d'abord dans le cas d'utilisation de la GUI puis abordons le cas d'une utilisation exclusive des packages R.

En utilisant l'interface graphique

L'utilisation de la GUI requiert donc l'organisation en *workspace(s)*. Ceux-ci peuvent être créés [manuellement](#)  ou via un script R, en s'inspirant du code suivant :

```
library("rjd3workspace")
# Création d'un workspace et d'un SA-Processing
my_ws <- jws_new()
jsap1 <- jws_sap_new(my_ws, "sap1")

# Début de remplissage avec Un SA-Item dans un SA-Processing
y <- rjd3toolkit::ABS$X0.2.09.10.M # serie brute
add_sa_item(jsap1, name = "serie_1", x = y, rjd3x13::x13_spec())

# Enregistrement du workspace sous forme de fichiers .xml lisibles par la GUI
save_workspace(my_ws, "C:/my_folder/workspace_test.xml")
```

Sélection et attribution d'un jeu de régresseurs de calendrier

Étape préalable à l'estimation, la sélection de régresseurs de calendrier consiste à déterminer quel jeu de régresseurs apporte la meilleure correction à chacune des séries. Pour cela on propose de suivre les étapes suivantes :

- créer un ensemble de jeux de régresseurs à tester, comme détaillé dans Attal-Toubert (2012) et illustré en [annexes](#)
- lancer une désaisonnalisation automatique (en gardant éventuellement un ensemble d'outliers pré-défini pertinent, par exemple ceux correspondant à la période covid) pour l'ensemble des jeux de régresseurs pre-sélectionnés
- noter l'efficacité relative de chaque jeu en pénalisant les effets de jours ouvrables résiduels
- choisir le jeu ayant la meilleure note

Une fonction du package {rjd3production} couvre ces étapes et renvoie une table où à chaque série en input correspond le meilleur jeu.

```
my_context <- create_insee_context()
my_context$variables <- my_context$variables[c("REG1", "REG1_LY", "REG6", "REG6_LY")]
td_table <- select_regs(ABS, context = my_context)
```

- attribuer à chaque série le jeu choisi en le déclarant dans la spécification correspondante

```
assign_td(td = td_table, jws = my_ws)
```

Personnalisation des spécifications

Au delà du choix d'un jeu de régresseurs de calendrier, chaque série est susceptible d'avoir des spécificités que le producteur souhaite imposer : des outliers pré-spécifiés, un schéma de décomposition ou une période d'estimation en sont les exemples les plus courants. Cette personnalisation peut se faire via l'interface graphique sur un nombre réduit de séries ou nécessiter l'exécution d'un script dans le cas d'un grand nombre de séries.

Dans l'exemple suivant on modifie un SA-Item (jsai) :

```
library(rjd3workspace)

# lecture du workspace
jws <- jws_open("C:/my_folder/workspace_test.xml")
```

```

# recuperation du sa-item dans le workspace (ici le 1er du 1er SA-processing)
jsap <- jws_sap(jws, 1L)
jsai <- jsap_sai(jsap, 1L)

spec0 <- get_domain_specification(jsai)

spec1_with_outliers <- rjd3toolkit::add_outlier(
  x = spec0,
  type = c("AO", "AO", "LS"),
  date = c("2020-02-01", "2020-03-01", "2014-08-01")
)

spec2_with_new_span <- rjd3toolkit::set_basic(spec1_with_outliers, type = "From", d0 = "2012-01-01")

spec3_with_td <- rjd3toolkit::set_tradingdays(
  spec2_with_new_span,
  option = "UserDefined",
  uservariable = c("REG1.group_1", "REG1.group_2"),
  test = "None"
)

set_domain_specification(jsap, 1L, spec3_with_td)

```

Ici nous attribuons des outliers pré-spécifiés série par série à un *workspace* entier.

```

library(rjd3production)
assign_outliers(outliers = outliers_table, jws = my_ws)

```

Les outliers doivent avoir au préalable été stockés dans un fichier “*.yaml” ou importés d’un *workspace* existant contenant les mêmes series.

Il est également possible de récupérer des spécifications X13-Arima ou Tramo-Seats provenant de logiciels autres et les rendre compatibles avec JDemetra+. Nous décrivons cette procédure de migration en [annexes](#) .

Estimation

Une fois les spécifications fixées, dans leur version initiale, avant d’éventuelles modifications à la vue des diagnostics, l’estimation peut être lancée manuellement dans l’interface graphique. Il est très courant d’automatiser cette étape avec le *Cruncher* qui exportera directement l’output (séries et diagnostics). On évite ainsi d’ouvrir la GUI en lançant le code ci-dessous :

```
library("rjwsacruncher")
cruncher_and_param(
  workspace = "C:/my_folder/my_ws.xml",
  rename_multi_documents = FALSE,
  policy = "complete"
)
```

Un sous-répertoire “output” est créé dans le répertoire du *workspace* avec les fichiers contenant les séries et le fichier contenant les diagnostics et les paramètres (demetra_m.csv). Le choix des objets à exporter se fait en posant des options dans R, le code correspondant peut-être consulté [ici](#) .

Dans l’exemple ci-dessus, la policy est “complete” (ou “concurrent”), ce qui est le seul choix possible pour une première estimation. Nous revenons en détails sur ce paramètre et les différentes politiques de rafraîchissement disponibles dans la [troisième partie](#) .

Si dans un *workspace* JDemetra+ il est possible d’organiser ses séries en plusieurs SA-Processings, il faut noter que le *Cruncher* agira systématiquement sur un *workspace* entier.

Le chemin vers l’emplacement physique des données doit être spécifié. Ce [lien est inscrit dans le workspace](#)  lorsque les données y sont affectées. Si, avant une actualisation de l’estimation, les données sont été déplacées, le chemin doit être mis à jour. Cela peut être fait avec les fonctions suivantes dans le cas d’un classeur excel :

```
# Chargement du workspace
my_ws <- jws_open("C:/my_folder/workspace_test.xml")

# Mise jour du chemin pour le 1er SA-Processing
spreadsheet_update_path(
  jws = my_ws,
  new_path = "C:/my_data/My_Series.xlsx",
  idx_sap = 1
)
# Sauvegarde du workspace (écriture du fichier sur le disque)
save_workspace(my_ws, "C:/my_folder/workspace_test.xml")
```

Le cas d’un fichier “.txt” ou “.csv” est illustré [ici](#) .

Bilan Qualité

On construit un bilan qualité consistant à attribuer à chaque série un score en fonction de diagnostics sélectionnés dans l’output de JDemetra+. Cela permettra de repérer rapidement

les séries ayant la plus mauvaise qualité statistique et ainsi de cibler l’expertise manuelle. Le score statistique peut, de plus, être pondéré par l’importance de la série en diffusion pour aboutir à une sélection encore plus pragmatique.

Le package {JDCruncher} produit ce bilan en s’appuyant sur les diagnostics et paramètres du fichier “demetra_m.csv” généré par la fonction `rjwsacruncher::cruncher_and_param`. Ce fichier peut être également directement généré via la GUI à l’aide de [menus déroulants](#) .

```
BQ ← JDCruncher::extract_QR("C:/Workspaces/my_ws/Output/SAProcessing-1/demetra_m.csv")
```

Les paramètres de l’extraction sont détaillés dans la documentation de la fonction `extractQR()` .

Un ensemble de diagnostics, détaillé en [annexes](#) , notamment relatifs à la saisonnalité résiduelle, aux effets de jours ouvrables résiduels et à la blancheur des résidus sont sélectionnés. On attribue à chacun d’entre eux une modalité (“Good”, “Incertain”, “Bad” ou “Severe”) selon des seuils qui peuvent être consultés avec l’option ci-dessous :

```
library("JDCruncher")
getOption("jdc_thresholds")
```

Ces seuils sont paramétrables avec la fonction `JDCruncher::set_threshold()` .

A chaque modalité correspond une note, ainsi (“Good”, “Incertain”, “Bad” ou “Severe”) deviendront (0, 1, 3, 5). Le score d’une série est la moyenne pondérée de ses notes et sa qualité d’autant plus basse que celui-ci est élevé. Les tests de saisonnalité résiduelle sur la série CVS et sur l’irrégulier ont la plus forte pondération. L’équation complète du score est détaillée en [en annexes](#) .

Cette équation de est également paramétrable avec l’argument `score_pond` de la fonction `compute_score()` .

Le résultat est exportable sous Excel, où il est par défaut organisé comme suit :

- une feuille avec les modalités des diagnostics pour chaque série
- une feuille avec les valeurs, dans lequel on retrouve le score, le score pondéré ainsi que les principales contributions au score

L’utilisateur peut enrichir ce fichier et y documenter l’expertise manuelle du *workspace*. Un tri par valeur décroissante du score (pondéré) donne le ciblage attendu.

Ci-dessous un extrait de l’export Excel du bilan qualité, par défaut :

series	score	f_residual	td_on_i	residuals_kurtosis	residuals_normality	residuals_independenc
X0.2.06.10.M	60	Good		Bad	Good	Bad
X0.2.11.10.M	250	Severe		Good	Bad	Good
X0.2.10.10.M	250	Severe		Good	Good	Good
X0.2.20.10.M	545	Severe		Uncertain	Bad	Uncertain
series	score	f_residual	td_on_i	residuals_kurtosis	residuals_normality	residuals_indepen
X0.2.06.10.M	60	0,0815		0	0,6324	0
X0.2.11.10.M	250	0		0,6252	0	0,6774
X0.2.10.10.M	250	0,0006		0,8809	0,1105	0,7377
X0.2.20.10.M	545	0		0,082	0	0,0167

Guidé par le bilan qualité, le producteur modifie les spécifications afin d'améliorer l'ajustement saisonnier, en général en utilisant l'interface graphique. Pour générer l'output final, notamment les séries, il peut relancer le *Cruncher*, non plus avec l'option `policy="complete"` mais en utilisant un rafraîchissement qui re-estime juste les coefficients du modèle de pré-ajustement (`"policy=arimaparameters"`) et garde l'ensemble des modifications de l'utilisateur. Nous donnons plus de détails sur ce point en [annexes](#) .

```
library("rjwsacruncher")
cruncher_and_param(
  workspace = "C:/my_folder/my_ws.xml",
  rename_multi_documents = FALSE,
  policy = "arimaparameters"
)
```

En utilisant uniquement des packages R

L'ensemble des étapes décrites ci-dessus est réalisable en s'affranchissant de la structure de *workspace* et en restant dans un environnement R.

Estimation

Dans ce cas on utilise directement les fonctions `rjd3x13::x13()`  (ou `rjd3tramoseats::tramoseats()` ) pour faire les estimations. Les fonctions d'estimation fonctionnent sur un objet TS, ce qui nécessite la mise en place de boucles ou de fonctions de type « lapply ».

```
# pour une série
mod <- x13(my_series, my_spec)
str(mod)
```

Diagnostics et spécifications

En R, on peut accéder aux mêmes diagnostics qu'à partir de la GUI et du *Cruncher*, en navigant dans l'objet produit par l'estimation (liste de listes), que l'on directement organiser en un bilan qualité personnalisé. Pour profiter de la mise en forme existante, il suffit de re-écrire un *workspace* crunchable et de re-générer le même bilan que dans l'approche avec GUI. Pour rester dans un environnement, la modification manuelle des spécification au vu des diagnostics se fera dans ce cas directement dans le script R, comme dans l'exemple ci-dessous :

```
spec0 <- rjd3x13::x13_spec()

spec1_with_outliers <- rjd3toolkit::add_outlier(
  x = spec0,
  type = c("A0", "A0", "LS"),
  date = c("2020-02-01", "2020-03-01", "2014-08-01")
)
```

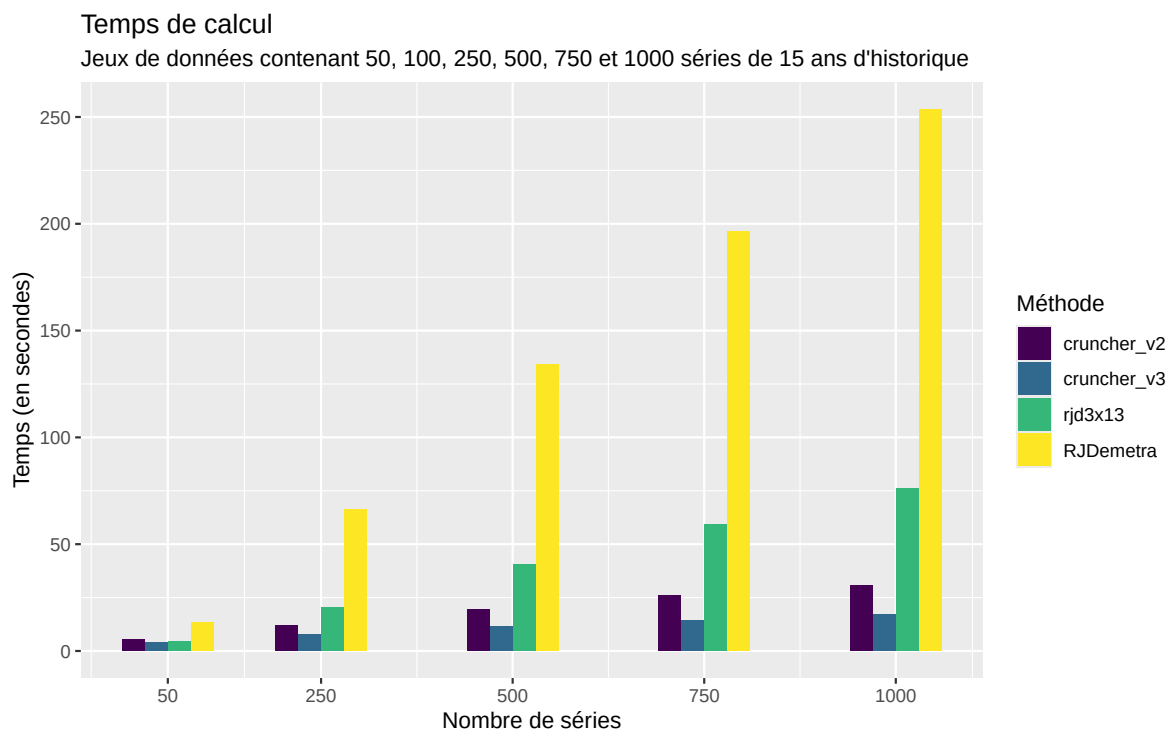
La boucle lecture des diagnostics - modification des paramètres est moins fluide, ce qui incite à utiliser l'approche avec Interface Graphique en cas d'expertise manuelle intensive. Pour mettre à jour les spécifications, au vu des diagnostics, dans R il faut modifier directement les scripts ce qui est moins rapide et plus sujet à erreur pour l'analyste, surtout non familier avec R. L'approche uniquement en R rend aussi plus complexe le stockage et le versionnage des spécifications.

Les diagnostics permettant de construire le score présenté ci-dessus peuvent être directement obtenus en R, ce qui permet de répliquer la procédure de sélective editing. Le code pour obtenir cet output figure en [annexes](#) .

L'approche R permet également de s'affranchir de tout problème de portabilité, même si la mise à jour du lien entre un *workspace* et les données est désormais simple et fiable.

Performance

Nous avons comparé les performances des deux approches. La graphique ci-dessous montre les temps de calcul pour la désaisonnalisation d'un jeu de données, avec détection d'outliers et estimation d'effets de calendrier avec un jeu de régresseurs présélectionné, en utilisant les packages R (v2 et v3) ou le *Cruncher* (v2 ou v3) :



Pour un nombre de séries proche du millier, l'estimation avec `{rjd3x13}` prend moins de trois minutes, ce qui reste très raisonnable pour la plupart des utilisateurs mais loin derrière le *Cruncher* qui accomplit le même processus en une vingtaine de secondes.

Une fois la chaîne de production mise en place, il est recommandé de valider ou ajuster les paramétrage tous les ans et d'adopter une stratégie de production infra-annuelle, en général mensuelle ou trimestrielle permettant de mettre à jour les données CVS-CJO lorsque de nouvelles données brutes sont disponibles. Cela fait l'objet de la partie suivante.

Partie 3 : Campagnes infra-annuelles et annuelles

Campagnes infra-annuelles

L'objectif d'une campagne infra-annuelle est de produire rapidement une série CVS-CJO actualisée lorsqu'un nouveau point brut est disponible et le passé récent de la série brute éventuellement révisé. La série désaisonnalisée comporte deux sources de révision : les données brutes mais aussi les modifications apportées aux modèles d'estimation des facteurs saisonniers finaux. Les guidelines sur l'ajustement saisonnier recommandent de minimiser ces révisions dites techniques (voir Chap 6). Une méthode commune permettant d'atteindre cet objectif est d'utiliser les facteurs saisonnier prévus, contenant l'effet de calendrier prévu, calculés

lors de la dernière campagne annuelle. Mais JDemetra+ propose aussi un continuum de méthodes de rafraîchissement permettant de desserrer progressivement les contraintes sur le pré-ajustement et de contrôler les révisions de la série linéarisée avant l'étape de décomposition, qui sera, elle, entièrement refaite. L'ensemble de ces refresh policies est présenté dans la [documentation](#), parmi celles-ci le choix “lastoutliers” est particulièrement recommandé. On s'autorise dans ce cas à re-identifier les outliers sur la dernière année. L'expertise manuelle dans le cadre infra-annuel va en effet se concentrer sur les révisions et donc les outliers, en évitant de trop nombreux changements de paramétrages. Ce rafraîchissement est possible depuis l'interface graphique, avec le *Cruncher* en spécifiant l'argument “policy” de la fonction `rjwsacruncher::cruncher_and_param()`, mais aussi directement en R avec les fonctions `rjd3x13::x13_refresh()` ou `rjd3tramoseats::tramoseats_refresh()`.

Table 4: Implémentation d'une refresh policy

Type de données	Outils
<i>workspace</i>	- GUI (Menus déroulants) - <code>rjwsacruncher::cruncher_and_param(... , policy=,)</code> - <code>rjd3workspace::jws_refresh()</code>
Objets TS en R	- <code>rjd3x13::x13_refresh()</code> - <code>rjd3tramoseats::tramoseats_refresh()</code>

L'utilisation de coefficients saisonniers projetés (S_f) nécessite simplement de les exporter à la fin du processus d'installation, puis à l'issue de chaque campagne annuelles, afin de pouvoir faire le calcul : $Y_{CVS-CJO} = Y_{brut} - S_f$ ou $Y_{CVS-CJO} = Y_{brut}/S_f$, dans le cas d'un modèle multiplicatif. Pour éventuellement prendre en compte une révision du passé récent de la série brute l'export des coefficients passés est également utile. Les séries des coefficients projetés peuvent être générées notamment par le *Cruncher* en le spécifiant dans les options, comme ci-dessous :

```
options(default_tsmatrix_series = c("sa", "sa_f", "s", "s_f"))
```

ou en R en définissant un user-defined output comme ci-dessous :

```
mod ← x13(y, userdefined = c("s", "s_f"))
str(mod$user_defined)
```

Campagnes annuelles

Une campagne annuelle consiste en la mise à jour des modèles d'estimation des coefficients saisonniers afin de contrôler que les paramètres utilisés permettent bien de produire des

séries désaisonnalisées ayant les propriétés requises (absence de saisonnalité résiduelle, absence d'effets de jours ouvrables résiduels...).

Cette mise à jour peut se faire par comparaison des modèles “en vigueur” (ou de référence) utilisés jusqu’au moment de la campagne avec une estimation automatique. Dans la plupart des cas l’estimation automatique sera contrainte par des paramètres passés comme la sélection de régresseurs de calendrier ou certains outliers pré-spécifiés, notamment ceux de la période covid. Pour comparer aisément, il peut être utile de calculer les scores du bilan qualité décrits dans le partie 2 pour le *workspace* “de référence” (ou “en vigueur”) et le *workspace* “automatique”. Un score inférieur pour l’estimation automatique conduit à préférer celle-ci, on pourra alors créer un nouveau *workspace* (“de travail”) en fusionnant le “en vigueur” et l’“automatique”, procédure détaillée en [annexes](#) .

Cette approche nécessite de travailler sur des séries brutes comparables, si celles-ci on subi des révisions importantes (rebasement, changement de source...) on en revient à une approche proche de l’installation de nouveau processus.

Principales étapes :

- mise jour du *workspace* de référence (“ref”) en utilisant les nouvelles données brutes et les paramètres statistiques de la précédente campagne annuelle, éventuellement modifié par la même stratégie qu’une campagne infra annuelle (par exemple: “lastoutliers”)
- estimation automatique (“auto”) sur les nouvelles données brutes, avec probablement le maintien de certains paramètres (jeu de régresseurs de calendrier, un partie ou tous les outliers “pré-spécifiés” par l’utilisateur...)
- calcul des scores des deux workspaces “ref” et “auto”
- fusion des workspaces “auto” et “ref” en un nouveau *workspace* “de travail” selon la valeur du score de chaque série
- édition d’un bilan qualité sur le *workspace* “travail”
- expertise manuelle avec sélective editing, comme dans la phase d’installation
- génération de l’output final avec le *Cruncher* comme dans la phase d’installation

Si l’on utilise uniquement des coefficients projetés lors des campagnes infra-annuelles, on peut actualiser le *workspace* de la précédente campagne annuelle avec un policy “last outliers” et se ramener au cas décrit ci-dessus.

Le détail de l’organisation des données et des extraits de code sont donnés dans la [vignette du package {rjd3production}](#) .

Impact numérique d'un changement de paramètres

Au vu des (mauvais) diagnostics, notamment repérés grâce au bilan qualité, le producteur peut intervenir, pour modifier les paramètres et améliorer la qualité statistique de l'ajustement. Par exemple, il modifie un jeu de régresseurs de calendrier et se débarrasse de l'effet de jours ouvrables résiduels qu'il avait auparavant constaté. Au delà de l'amélioration des résultats de tests statistiques, il est souvent intéressant de visualiser l'impact numérique d'une modification de paramètres ou d'une re-estimation, ce qui permet de quantifier et visualiser les révisions, mais aussi de valider certains paramétrages, notamment le choix d'outliers à "dire d'expert". Exporter les séries produites est une solution, mais si l'on souhaite faire ces comparaisons avant d'avoir terminé de travailler sur un *workspace*, il est possible d'utiliser la fonction `compare()` [🔗](#) du package `{rjd3production}`, qui évite de générer des outputs intermédiaires. Celle-ci permet de lire directement les séries dans deux ou plusieurs workspaces correspondant chacun à une version des paramètres ou un état de l'estimation, comme le montre le code suivant :

```
df <- compare(path_ws_ref, path_ws_work)
run_app(df)
```

La fonction démarre une application *Shiny* qui fournit une comparaison graphique, ci-dessous la tendance finale (d12), et la possibilité de télécharger les valeurs des séries pour analyser les révisions.

Comparateur de séries

Choisir un SAI :

X0.2.09.10.M ▼

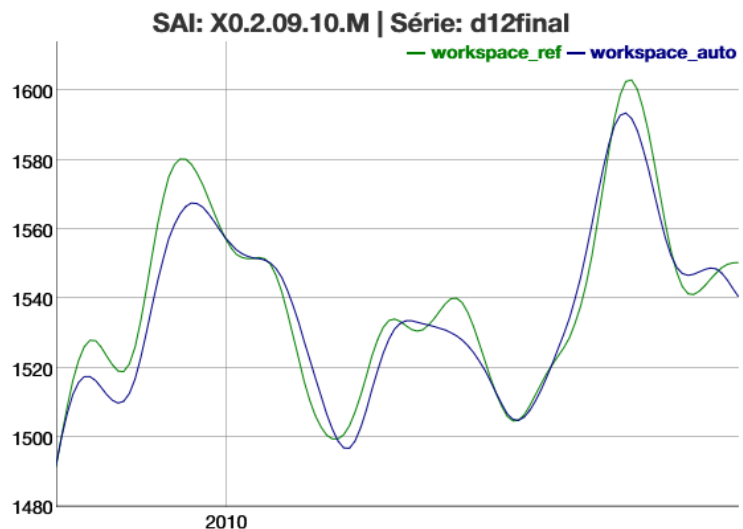
Choisir une série :

d12final ▼

☒ Filtrer par SAI

☒ Filtrer par série

📄 Exporter le tableau en CSV



Conclusion

Garantir la qualité statistique des séries désaisonnalisées requiert de mettre en place un processus de production permettant, à la lumière des diagnostics, de personnaliser facilement et à grande échelle les paramètres d'estimation. L'utilisation du logiciel *open source* JDemetra+ et de ses outils R permet d'atteindre cet objectif en offrant de nombreuses possibilités d'automatisation tout en rendant accessible une expertise manuelle fine. Nous avons montré que le choix d'intégrer l'interface graphique dans une chaîne de production est structurant pour l'organisation des données. Le recours à une expertise manuelle et la performance devraient en être les principales raisons. Toutefois, une utilisation exclusive de R, sans *workspace*, n'exclut pas un recours ad hoc à la GUI ou au *Cruncher*. L'organisation des opérations et l'utilisation des outils telles que décrites dans cet article ne sont que des suggestions, qui nous l'espérons fourniront une première réponse pratique aux nombreux producteurs de séries cvs-cjo désireux d'utiliser JDemetra+.

Afin de dialoguer avec leurs utilisateurs, ceux-ci ont fréquemment besoin d'outils de reporting décomposant les effets de la désaisonnalisation sur les évolutions d'une série donnée, pour mettre notamment en valeur les différentes contributions aux révisions. De tels outils sont souvent très *ad hoc* mais il existe des packages ou plug-in, liés à JDemetra+, souvent encore en version 2, qui permettent d'obtenir rapidement ce genre d'informations. Il serait intéressant de donner suite à cet article en proposant une revue des outils de reporting *open source* et de mettre à disposition un package R permettant de tirer pleinement profit de la version 3 de JDemetra+.

Références

- Attal Toubert, Ketty. 2012. « Régresseurs pour effets de calendrier: Comment les construire, comment les choisir ». *XIèmes Journées de Méthodologie Statistique de l'Insee*.
- Bell, William R, et Steven C Hillmer. 1984. « Issues involved with the seasonal adjustment of economic time series ». *Journal of Business & Economic Statistics* 2 (4): 291-320.
- Eurostat. 2024. « ESS Guidelines on Seasonal Adjustment ». Eurostat Methodologies; Working Papers, European Commission. <https://doi.org/10.2785/705102> .
- Gomez, Victor, et Agustin Maravall. 1996. « Programs TRAMO (Time series regression with arima noise, missing observations, and outliers) and SEATS (Signal extraction in arima time series). Instructions for the User ». *Documento de Trabajo* 9628: 56.
- Grudkowska, Sylwia, Dario Buono, Jean Palate, et Wojciech Ciebiera. 2013. « Advanced tools for time series analysis and seasonal adjustment in the new jdemetra+ ». *JSM Proceedings Paper*.
- Kirchner, et alli. 2018. « Quality Measures and Reporting for Seasonal Adjustment ». *Handbook on Seasonal Adjustment*. ec.europa.eu/eurostat/web/products-manuals-and-guidelines/-/KS-GQ-18-001.
- Smyk, A, et K Webel. 2024. « Vers une désaisonnalisation des séries temporelles infra-mensuelles avec JDemetra+ ». Institut National de la Statistique et des Etudes Economiques.
- Smyk, et alli. 2026. « Online JD+ documentation ». 2026. <https://jdemetra-new-documentation.netlify.app/>.
- US-Census-Bureau. 2015. « The X-13ARIMA-Seats seasonal adjustment program ». 2015. <https://www.census.gov/srd/www/x13as>.

Annexes

Correction des effets de calendrier

Deux types de procédures sont possibles pour créer des régresseurs de calendrier : la création de régresseurs personnalisés comme nous le montrons ci-dessous ou l'utilisation de régresseurs prédéfinis dans JDemetra+, ce qui nécessite également la définition d'un calendrier de référence. La version 3.x de la GUI permet d'opérer une [sélection](#)  fondée sur des tests statistiques parmi les jeux de régresseurs prédéfinis.

En lieu et place d'un calendrier [national simple](#) , JDemetra+ permet de créer des [calendriers "chaînés"](#)  ou ["composites"](#)  avec la GUI ou [rjd3toolkit](#) .

Création d'un calendrier de référence

Pour créer un calendrier français suivant les jours fériés français, on peut utiliser la fonction `create_french_calendar()`  du package `{rjd3production}` :

```
cal_FR <- create_french_calendar()
```

Cette fonction est facilement adaptable à n'importe quel calendrier national.

Génération des jeux régresseurs

A partir du calendrier français, il est possible de générer des régresseurs de calendriers suivant, par exemple, les regroupements usuels utilisés à l'Insee :

```
regs <- create_insee_regressors(start = c(2000, 1), frequency = 12, length = 360)
sets <- create_insee_regressors_sets(start = c(2000, 1), frequency = 12, length = 240)
names(sets)
```

```
[1] "REG1"      "REG2"      "REG3"      "REG5"      "REG6"      "LY"        "REG1_LY"
[8] "REG2_LY"   "REG3_LY"   "REG5_LY"   "REG6_LY"
```

Bilan Qualité : diagnostics et code R

Les diagnostics sélectionnés par défaut, lorsque l'on utilise X-13-Arima⁴, sont les suivants :

Tableau A1 : Diagnostics

⁴Lorsque l'on utilise Tramo-Seats cela fonctionne aussi mais dans la version les diagnostics spécifiques à X-13-Arima sont simplement supprimés et pour l'instant non remplacés

Diagnostic	Nom de variable dans le fichier demetra m.csv (Cruncher ou GUI)	Accès avec rjd3x13
Test de skewness sur les résidus du modèle Reg-Arima	residuals.skewness.3	m_sa\$result\$preprocessing\$diagnostics\$skewness
Test de Ljung-Box sur les résidus du modèle Reg-Arima	residuals.lb2.3	m_sa\$result\$preprocessing\$diagnostics\$lb2
Test de normalité (Doornik-Hansen) sur les résidus du modèle Reg-Arima	residuals.doornikhansen.3	m_sa\$result\$preprocessing\$diagnostics\$doornikhansen
QS-Test de saisonnalité résiduelle sur la série cvs (sa) finale	diagnostics.seas-sa-qs.2	m_sa\$result\$diagnostics\$seas.qstest.sa
QS- Test de saisonnalité résiduelle sur l'irrégulier (i) final	diagnostics.seas-i-qs.2	m_sa\$result\$diagnostics\$seas.qstest.i
F-Test de saisonnalité résiduelle sur la série cvs (sa) finale	diagnostics.seas-sa-f.2	m_sa\$result\$diagnostics\$seas.ftest.sa
F-Test de saisonnalité résiduelle sur l'irrégulier (i) final	diagnostics.seas-i-f.2	m_sa\$result\$diagnostics\$seas.ftest.i
F-Test d'effets de jours ouvrables résiduels sur la série cvs (sa) finale	diagnostics.td-sa-last.2	m_sa\$result\$diagnostics\$td.ftest.sa
F-Test d'effets de jours ouvrables résiduels sur l'irrégulier (i) final	diagnostics.td-i-last.2	m_sa\$result\$diagnostics\$td.ftest.i
Test d'égalité des moyennes entre prévisions et échantillon	diagnostics.fcast-outsamp-mean.2	m_sa\$user_defined\$`diagnostics.fcast-outsamp-mean`
Test d'égalité des variances entre prévisions et échantillon	diagnostics.fcast-outsamp-variance.2	m_sa\$user_defined\$`diagnostics.fcast-outsamp-variance`
Décomposition X-11 : statistique Q-M2	m-statistics.q-m2	m_sa\$result\$mstats\$qm2
Décomposition X-11 : statistique M7	m-statistics.m7	m_sa\$result\$mstats\$m7

En R les diagnostics sont extraits de l'objet "m_sa" contenant tous les résultats de l'estimation et obtenu avec le code ci-dessous.

```
m_sa <- x13(my_series, my_spec, userdefined = c("diagnostics.fcast-outsamp-mean", "diagnostics.fcast-outsamp-variance"), str(m_sa))
```

Plus de détails sur les tests sont disponibles dans la [documentation](#) 

Équation de score

$$\begin{aligned}
 score_total = & 30 * score_qs_residual_s_on_sa \\
 & + 30 * note_f_residual_s_on_sa \\
 & + 30 * note_f_residual_td_on_sa \\
 & + 20 * note_f_residual_td_on_i \\
 & + 20 * note_qs_residual_sa_on_i \\
 & + 20 * note_f_residual_sa_on_i \\
 & + 15 * note_oos_mean \\
 & + 10 * note_oos_mse \\
 & + 15 * note_residuals_independency \\
 & + 5 * note_residuals_skewness \\
 & + 5 * note_residuals_homoskedasticity \\
 & + 5 * note_q_m2 \\
 & + 5 * note_m7
 \end{aligned}$$

Campagne annuelle

Création du workspace travail

Le **WS_travail** est une fusion entre le **WS_ref** et le **WS_auto**. On calcule un score pour les deux workspaces et on récupère pour chaque série, le SA-Item dans le *workspace* ayant le score le plus faible entre WS_auto et WS_ref.

Dans un processus de campagne annuelle on utilisera la fonction `transfer_sa_item()`  de {rjd3workspace} pour fusionner **WS_ref** et **WS_auto**.

```
WS_auto <- jws_open(file = "./WS/WS_auto.xml")

# Sauvegarder WS_ref comme version initiale du workspace de travail
# Le WS de travail est une copie du WS ref
WS_work <- jws_open(file = "./WS/WS_ref.xml")

sap1_auto <- jws_sap(jws = WS_auto, idx = 1)
sap1_work <- jws_sap(jws = WS_work, idx = 1)

# mettre à jour les sa-item du workspace de travail avec ceux du ws_auto

if (score_auto > score_ref + 30) {
  transfer_sa_item(
    jsap_from = sap1_auto,
    jsap_to = sap1_work,
    selected_sa_items = series_name
  )
}

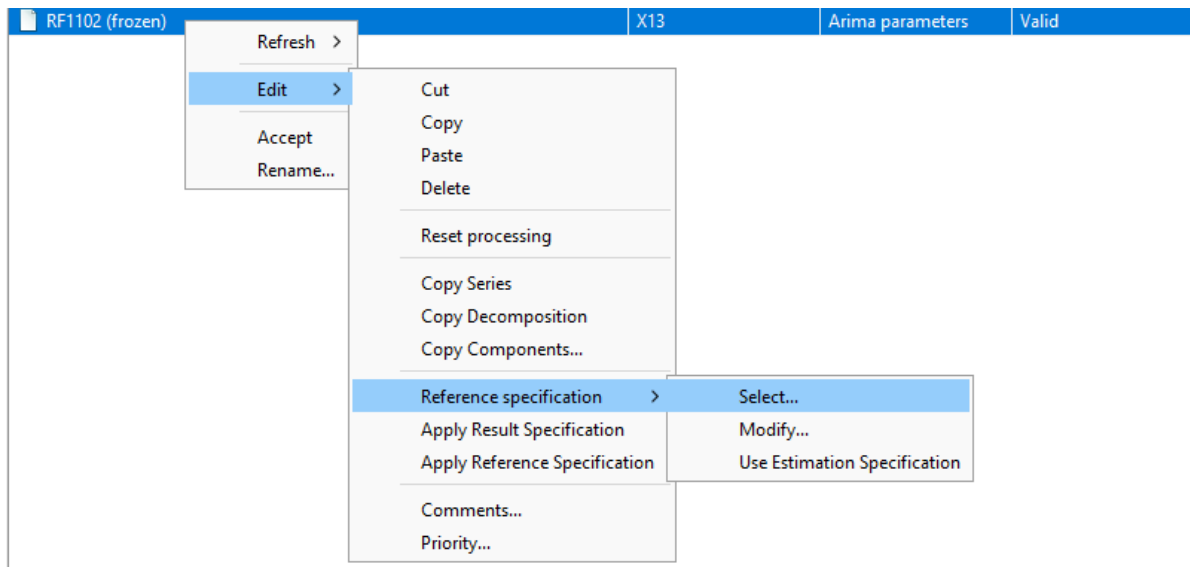
save_workspace(jws = WS_work, file = "./WS/WS_work.xml")
```

Compléments sur les spécifications

Modification

- Reference Specification

Modification dans la GUI:



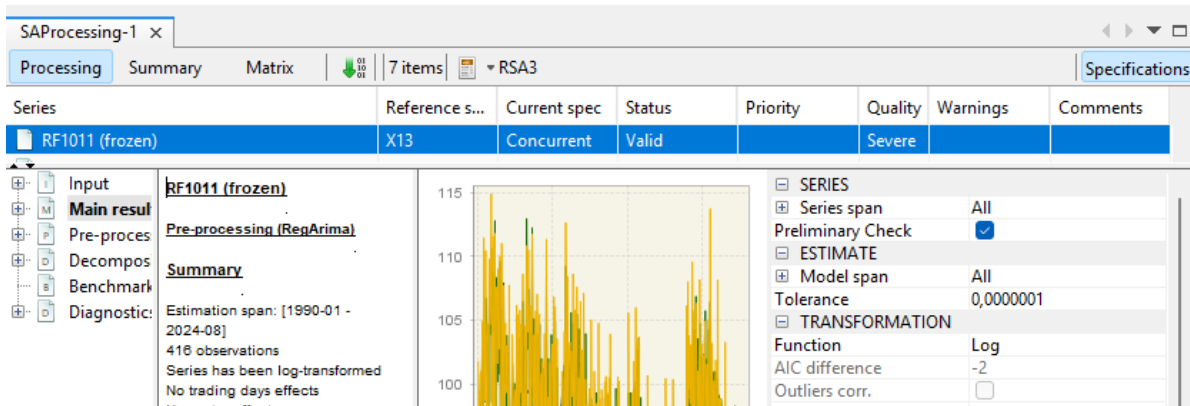
En utilisant ce même menu on peut appliquer la Reference Specification ou la Result Specification à une série.

Dans le package `rjd3workspace`, la Reference Specification est appelée Domain Specification :

```
library("rjd3workspace")
set_domain_specification(jsap, 1L, new_domain_spec)
```

- Estimation Specification

Modification dans la GUI, on utilise le bouton Specification à droite :



Dans le package `rjd3workspace`, l'Estimation Specification est simplement appelée Specification :

```
library("rjd3workspace")
set_specification(jsap, 1L, new_estimation_spec)
```

- Result Specification

Le résultat de l'estimation peut être appliqué comme indiqué ci-dessous.

```
library("rjd3workspace")

jws_compute(jws)
read_workspace(jws, compute = TRUE)
read_sap(jsap)
read_sai(jsai)
```

Comportement des politiques de rafraîchissement

Une politique de rafraîchissement a pour objectif de lever toutes ou une partie des contraintes de l'Estimation Specification et de revenir à la Reference Specification.

- la policy “concurrent” revient à la Reference Specification en effaçant les paramètres user-defined non écrits dans la domain spec (les modifications via la fenêtre de spécification à droite sont supprimées). Notamment, elle efface les outliers pré-spécifiés dans l'Estimation Specification, l'ensemble des outliers sera re-identifié
- la policy “lastoutliers” efface les outliers pré-spécifiés dans l'Estimation Specification sur la dernière année (ou autre span défini), une re-identification sera faite sur cette période.

Ainsi l'utilisateur peut faire la différence entre des paramétrages permanents (Reference Specification) et transitoires (Estimation Specification).

Exemple

Voici une procédure courante actualisation des données dans le cas d'une campagne infra-annuelle avec génération de l'output en utilisant le *Cruncher*. Attention, il faut enregistrer le *workspace* avant de faire un “refresh” (via la GUI ou le *Cruncher*).

- De nouvelles données brutes sont disponibles (nouveaux points et révision du passé récent).
- On utilise un “refresh last outliers” pour bénéficier de la détection automatique sur la dernière année

- On vérifie les résultats, les révisions et pour certaines séries importantes, on souhaite revoir la détection automatique
- On ouvre l'interface graphique et l'on l'on modifie l'Estimation Specification d'une série via la fenêtre située à droite en ajoutant un outlier (Out_1) dans la dernière année
- On applique et sauvegarde ces modifications dans la GUI
- On doit régénérer un output avec le *Cruncher* (export des séries finales)
- si le *Cruncher* utilise "policy=lastoutliers" de nouveau : il fait retour à la Reference Specification pour la dernière année, Out_1 est perdu
- si l'on souhaite conserver Out_1 , on utilise l'option policy = « arimaparameters », l'identification ne change pas, seuls les coefficients sont ré-estimés
- pour conserver Out_1 et ré-identifier avec "lastoutliers" de nouveau, on doit écrire Out_1 dans la Reference Specification.

Différences entre la version 2 et la version 3

Dans JDemetra+ version 2.x, il n'y avait pas d'Estimation Specification. Les paramétrages fixés par l'utilisateur le sont donc forcément dans la Reference Specification et ne sont donc jamais effacés par une politique de rafraîchissement.

On peut ouvrir en version 3 des workspaces créés en version 2, mais le retour en version 2 ne sera plus possible.

Copie des spécifications lorsqu'un *workspace* version 2 est ouvert en version 3 :

- Reference Specification copiée dans Reference Specification et dans Estimation Specification
- Result Specification dans Result Specification

Structures de données

Les workspaces sont organisés en SA-Processing et SA-Item.

SA-Processing

Un SA-Processing (SAP) est représenté par un fichier .xml. Il contient des SA-Item et des spécifications de références définies par l'utilisateur.

SA-Item

Un SA-Item contient les informations relatives à une série donnée :

- les données brutes
- les spécifications (`pointSpecification`, `estimationSpecification`, `Reference Specification`)
- les métadonnées

Metadonnées

Les métadonnées (voir balise dans le fichier `.xml` correspondant au SA-Processing) contiennent :

- le type de source des données brutes
- l’“id” du fichier contenant les données brutes (chemin et paramètres) qui sera utilisé rafraîchir l’estimation
- d’éventuels commentaires sur la série (permettant en général de justifier un choix de paramètres)

Modelling context

Le modelling context contient les calendriers, variables et régresseurs externes nécessaires aux estimations faites dans le *workspace*.

Migration vers JDemetra+

Il est possible de convertir des spécifications X-13 utilisées dans le logiciel X-13-Arima-Seats de l’US Census Bureau en des spécifications X13-Arima de JDemetra+.

Cette opération se fait avec l’interface graphique avec le plug-in [SpecParser](#) , utilisable uniquement en version 2. Le *workspace* qui contiendra ces spécifications pourra être ouvert en version 3. La documentation du plug-in se trouve [ici](#)  et l’ensemble des étapes de la migration est décrit [ici](#)  dans la documentation de JDemetra+.